



Python Programming Fundamentals

Sorting Algorithms

Programming Fundamentals Team

SETU

2026-06-12



Outline

1. Why Sorting?
2. Bubble Sort
3. Bubble Sort Trace
4. Selection Sort
5. Python's Built-in Sort — Timsort
6. Time Complexity
7. Sorting Objects



Outline

1. Why Sorting?

2. Bubble Sort

3. Bubble Sort Trace

4. Selection Sort

5. Python's Built-in Sort — Timsort

6. Time Complexity

7. Sorting Objects



1. Why Sorting?

Sorted data is essential for:

- **Binary search** (requires sorted input)
- **Displaying** data in a meaningful order
- **Finding** min/max efficiently
- **Removing duplicates** (adjacent equal items are easy to spot)

We study two classic algorithms (Bubble Sort, Selection Sort) to understand sorting fundamentals, then use Python's built-in sort in practice.



Outline

1. Why Sorting?
- 2. Bubble Sort**
3. Bubble Sort Trace
4. Selection Sort
5. Python's Built-in Sort — Timsort
6. Time Complexity
7. Sorting Objects



2.1 Compare and swap adjacent pairs

```
def bubble_sort(lst: list) -> list:
    data = lst.copy()          # don't modify original
    n = len(data)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if data[j] > data[j + 1]:
                data[j], data[j + 1] = data[j + 1], data[j]
    return data

nums = [64, 25, 12, 22, 11]
print(bubble_sort(nums))      # [11, 12, 22, 25, 64]
```

Each pass “bubbles” the largest unsorted element to its correct position.



Outline

1. Why Sorting?
2. Bubble Sort
- 3. Bubble Sort Trace**
4. Selection Sort
5. Python's Built-in Sort — Timsort
6. Time Complexity
7. Sorting Objects



3. Bubble Sort Trace

Starting: [64, 25, 12, 22, 11]

Pass 1 (i=0): compare adjacent pairs

[64, 25] → swap → [25, 64, 12, 22, 11]

[64, 12] → swap → [25, 12, 64, 22, 11]

[64, 22] → swap → [25, 12, 22, 64, 11]

[64, 11] → swap → [25, 12, 22, 11, 64] ← 64 in place

Pass 2 (i=1): [25, 12, 22, 11, 64]

→ [12, 25, 22, 11, 64]

→ [12, 22, 25, 11, 64]

→ [12, 22, 11, 25, 64] ← 25 in place



Outline

1. Why Sorting?
2. Bubble Sort
3. Bubble Sort Trace
- 4. Selection Sort**
5. Python's Built-in Sort — Timsort
6. Time Complexity
7. Sorting Objects



4.1 Find minimum, move to front

```
def selection_sort(lst: list) -> list:
    data = lst.copy()
    n = len(data)
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            if data[j] < data[min_idx]:
                min_idx = j
        if min_idx != i:
            data[i], data[min_idx] = data[min_idx], data[i]
    return data
```

```
nums = [64, 25, 12, 22, 11]
print(selection_sort(nums))    # [11, 12, 22, 25, 64]
```

Each pass finds the smallest remaining element and places it at the front.



Outline

1. Why Sorting?
2. Bubble Sort
3. Bubble Sort Trace
4. Selection Sort
- 5. Python's Built-in Sort — Timsort**
6. Time Complexity
7. Sorting Objects



5. Python's Built-in Sort — Timsort

```
# sort() – in place
numbers = [3, 1, 4, 1, 5, 9]
numbers.sort()
print(numbers)    # [1, 1, 3, 4, 5, 9]

# sorted() – returns new list
words = ["banana", "apple", "cherry"]
print(sorted(words))    # ['apple', 'banana', 'cherry']

# With key=
products.sort(key=lambda p: p.get_price())

# Reverse
products.sort(key=lambda p: p.get_price(), reverse=True)
```



5. Python's Built-in Sort — Timsort

Python's sort is **Timsort** — a hybrid of merge sort and insertion sort, $O(n \log n)$ in all cases.



Outline

1. Why Sorting?
2. Bubble Sort
3. Bubble Sort Trace
4. Selection Sort
5. Python's Built-in Sort — Timsort
- 6. Time Complexity**
7. Sorting Objects



6. Time Complexity

Algorithm	Best	Average	Worst
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Python sort()	$O(n)$	$O(n \log n)$	$O(n \log n)$

For real code: always use Python's built-in sort.

For learning: implement Bubble/Selection to understand the concept.

$n=1000$: $O(n^2) \rightarrow 1,000,000$ operations. $O(n \log n) \rightarrow 10,000$. The difference matters!



Outline

1. Why Sorting?
2. Bubble Sort
3. Bubble Sort Trace
4. Selection Sort
5. Python's Built-in Sort — Timsort
6. Time Complexity
- 7. Sorting Objects**



7. Sorting Objects

```
from product import Product
```

```
products = [  
    Product("Butter", 3.00, 104),  
    Product("Apple", 1.99, 101),  
    Product("Milk", 1.20, 103),  
]
```

```
# By price
```

```
by_price = sorted(products, key=lambda p: p.get_price())
```

```
# By name
```

```
by_name = sorted(products, key=lambda p: p.get_name())
```

```
# Multiple criteria: first by category, then by price
```

```
products.sort(key=lambda p: (p.get_category(), p.get_price()))
```



Thanks for Watching - Any questions?

